The Complete .NET Full Stack Software Engineering Textbook

An Enterprise-Grade Curriculum Blueprint from Database Persistence to Cloud Pipelines

Created for .NET FSD Training Track

May 21, 2026

Contents

Chapter 1

Enterprise Design Principles & Diagnostics

1.1 The SOLID Design Principles

Writing robust, scalable, and maintainable applications requires transitioning from unstructured OOP to structured design theory. The SOLID principles present five core guidelines to decouple classes and design extensible systems.

1.1.1 Single Responsibility Principle (SRP)

A class should have one, and only one, reason to change. This means a component should focus on a single piece of business capability. For example, a `StudentService` class should calculate student grades, but it should not handle saving the data directly to a flat disk or writing Diagnostic console logs.

1.1.2 Open/Closed Principle (OCP)

Software entities (classes, modules, functions) should be **open for extension, but closed for modification**. You should be able to alter a class's behavior without editing its original source code. This is achieved through abstractions (interfaces or inheritance).

1.1.3 Liskov Substitution Principle (LSP)

Subtypes must be substitutionable for their base types without altering the correctness of the program. If Class B is a subclass of Class A, we should be able to pass an instance of B to any method expecting A without throwing unexpected exceptions (e.g., `NotImplementedException`).

1.1.4 Interface Segregation Principle (ISP)

No client should be forced to implement interfaces and methods it does not use. Large interfaces should be broken down into smaller, highly specialized ones.

1.1.5 Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions.

> **Trainer Note:** When teaching SOLID, always contrast an anti-pattern with a refactored solution. Show how violating DIP makes unit testing impossible because low-level database operations cannot be mocked.

1.2 Logging and Telemetry with log4net

Enterprise systems require real-time diagnostics. The Apache `log4net` framework allows developers to log messages to various outputs (Appenders) using a severity hierarchy:

$$\text{DEBUG} < \text{INFO} < \text{WARN} < \text{ERROR} < \text{FATAL}$$

Here is a typical production-grade configuration block for a rolling log file appender:

```
<log4net>
  <appender name="RollingFileAppender" type="log4net.Appender.
   RollingFileAppender">
    <file value="logs/app.log" />
    <appendToFile value="true" />
    <rollingStyle value="Date" />
    <datePattern value="yyyy-MM-dd'.log'" />
    <layout type="log4net.Layout.PatternLayout">
      <conversionPattern value="%date [%thread] %-5level %logger - %message
   %newline" />
    </layout>
  </appender>
  <root>
    <level value="DEBUG" />
    <appender-ref ref="RollingFileAppender" />
  </root>
</log4net>
```

Listing 1.1: log4net.config Configuration Example

Implementing logger instantiation inside a class:

```
using log4net;
using System;

public class Program
{
    private static readonly ILog log = LogManager.GetLogger(typeof(Program)
   );

    public static void Main(string[] args)
    {
        log.Info("Application started successfully.");
        try
        {
            int x = 10, y = 0;
            int result = x / y;
        }
        catch (DivideByZeroException ex)
        {
            log.Error("An unhandled math exception occurred.", ex);
        }
    }
}
```

Listing 1.2: log4net Engine Initialization in C#

Chapter 2

ASP.NET Core Web API Architecture

2.1 The Middleware Pipeline

A middleware pipeline is a bidirectional chain of components that process incoming HTTP requests and outgoing HTTP responses. Each component can inspect, alter, or pass the request forward using a `RequestDelegate`, or short-circuit the execution flow to return early.

HTTP Request Pipeline Execution Order

$\rightarrow$ [Exception Handler] $\rightarrow$ [HTTPS Redirection] $\rightarrow$ [Routing] $\rightarrow$ [Auth] $\rightarrow$ [Endpoint/Controller]

$\leftarrow$ [Exception Handler] $\leftarrow$ [HTTPS Redirection] $\leftarrow$ [Routing] $\leftarrow$ [Auth] $\leftarrow$ [Response Out]

2.1.1 Custom Middleware Implementation

Below is a custom middleware component that measures execution timing and injects a custom HTTP response header containing the duration.

```
using Microsoft.AspNetCore.Http;
using System.Diagnostics;
using System.Threading.Tasks;

public class RequestTimingMiddleware
{
    private readonly RequestDelegate _next;

    public RequestTimingMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        var stopwatch = Stopwatch.StartNew();

        // Let the request flow down the pipeline
        await _next(context);

        stopwatch.Stop();
        context.Response.Headers["X-Response-Time-ms"] = stopwatch.
   ElapsedMilliseconds.ToString();
```

```
    }
}
```

Listing 2.1: Request Timing Middleware

To keep the pipeline declaration clean, create an extension method wrapper:

```
using Microsoft.AspNetCore.Builder;

public static class MiddlewareExtensions
{
    public static IApplicationBuilder UseRequestTiming(this
    IApplicationBuilder app)
    {
        return app.UseMiddleware<RequestTimingMiddleware>();
    }
}
```

Listing 2.2: Middleware Extension Wrapper

2.2 Unified API Response & Global Exception Handling

Consistent response payloads simplify client consumption. In production APIs, you should never return unhandled raw stack traces or inconsistent models.

2.2.1 The Generic API Response Wrapper

All HTTP endpoints must output a unified structure matching the following contract:

```
public class ApiResponse<T>
{
    public bool Status { get; set; }
    public string Message { get; set; }
    public T Data { get; set; }
    public List<string> Errors { get; set; }

    public ApiResponse()
    {
        Errors = new List<string>();
    }
}
```

Listing 2.3: Generic ApiResponse Schema

2.2.2 Centralized Global Error Middleware

This custom component acts as a high-level catch-all block, logging errors via diagnostics tools and generating an RFC-compliant response envelope.

```
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Logging;
using System;
using System.Net;
using System.Text.Json;
using System.Threading.Tasks;

public class ExceptionMiddleware
```

```
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionMiddleware> _logger;

    public ExceptionMiddleware(RequestDelegate next, ILogger<
    ExceptionMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "An unhandled exception occurred in the
    application pipeline.");
            await HandleExceptionAsync(context, ex);
        }
    }

    private static Task HandleExceptionAsync(HttpContext context, Exception
     ex)
    {
        context.Response.ContentType = "application/json";
        context.Response.StatusCode = (int)HttpStatusCode.
    InternalServerError;

        var payload = new ApiResponse<object>
        {
            Status = false,
            Message = "An unexpected error occurred. Please contact system
    administrators.",
            Errors = { ex.Message }
        };

        var json = JsonSerializer.Serialize(payload, new
    JsonSerializerOptions { PropertyNamingPolicy = JsonNamingPolicy.
    CamelCase });
        return context.Response.WriteAsync(json);
    }
}
```

Listing 2.4: Centralized Exception Handler Middleware

2.3 API Versioning & Swagger Security

APIs must evolve without disrupting existing consumers. This is managed via API Versioning and documented with customized Swagger interfaces.

2.3.1 API Versioning Implementation

Configure programmatic versioning using parameters inside `Program.cs`:

```
builder.Services.AddApiVersioning(options =>
{
    options.AssumeDefaultVersionWhenUnspecified = true;
    options.DefaultApiVersion = new ApiVersion(1, 0);
    options.ReportApiVersions = true; // Injects headers like "api-supported-versions"
});
```

Listing 2.5: API Versioning Configuration

Apply version specifications on controllers:

```
[ApiVersion("1.0", Deprecated = true)]
[Route("api/v{version:apiVersion}/products")]
public class ProductsV1Controller : ControllerBase
{
    [HttpGet]
    public IActionResult Get() => Ok(new[] { "Legacy Product A", "Legacy Product B" });
}
```

Listing 2.6: Versioned Endpoint Example

2.3.2 Swagger JWT Security Integration

To lock endpoints and enable testing JWT authorization tokens via the Swagger UI dashboard:

```
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "Enterprise Gateway API", Version = "v1" });

    var securityScheme = new OpenApiSecurityScheme
    {
        Name = "JWT Authentication",
        Description = "Enter JWT Bearer token **_only_**",
        In = ParameterLocation.Header,
        Type = SecuritySchemeType.Http,
        Scheme = "bearer",
        BearerFormat = "JWT",
        Reference = new OpenApiReference
        {
            Id = JwtBearerDefaults.AuthenticationScheme,
            Type = ReferenceType.SecurityScheme
        }
    };
    c.AddSecurityDefinition(securityScheme.Reference.Id, securityScheme);
    c.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        { securityScheme, Array.Empty<string>() }
    });
});
```

Listing 2.7: Swagger JWT Bearer Configuration

Chapter 3

Data Persistence with Entity Framework Core

3.1 Conventions vs. Fluent API

By default, Entity Framework Core applies logical conventions (e.g., class names pluralize into tables, properties named `Id` become primary keys, nullable properties translate to nullable columns). In complex systems, these assumptions must be overridden using the **Fluent API**.

Table 3.1: EF Core Mapping Comparisons

Feature	Default Convention	Fluent API Override
Table Name	Class Name Pluralized (`Students`)	Explicit Assignment (`tbl_students`)
Schema Name	`dbo`	Custom Schema Name (`admin`)
Primary Key	`Id` or `StudentId`	Independent Mapping (`HasKey(s => s.Student`
Nullability	Derived from C# Optional Types (`?`)	Explicit Overrides (`IsRequired()`)
Foreign Key	Derived from Navigation Properties	Explicit Mapping (`HasForeignKey()`)

3.1.1 Advanced Database Mapping Implementation

Here is a secure configuration block using the Fluent API that completely decouples model property names from the actual database structure.

```
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(
    options) { }

    public DbSet<Student> Students { get; set; }
    public DbSet<Course> Courses { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Course>(entity =>
        {
            entity.ToTable("tbl_courses", "admin");
            entity.HasKey(c => c.CourseKey);
```

```
            entity.Property(c => c.CourseKey).HasColumnName("course_id");
            entity.Property(c => c.Title)
                  .HasColumnName("course_title")
                  .HasMaxLength(100)
                  .IsRequired();
        });

        modelBuilder.Entity<Student>(entity =>
        {
            entity.ToTable("tbl_students", "admin");
            entity.HasKey(s => s.StudentKey);
            entity.Property(s => s.StudentKey).HasColumnName("student_id");

            entity.Property(s => s.FullName)
                  .HasColumnName("full_name")
                  .HasMaxLength(150)
                  .IsRequired();

            entity.Property(s => s.EmailAddress)
                  .HasColumnName("email")
                  .HasMaxLength(200)
                  .IsRequired(false);

            // Establish 1:N Relationship
            entity.HasOne(s => s.Course)
                  .WithMany(c => c.Students)
                  .HasForeignKey(s => s.CourseRefId)
                  .OnDelete(DeleteBehavior.Cascade);

            entity.HasIndex(s => s.EmailAddress)
                  .HasDatabaseName("IX_Student_Email")
                  .IsUnique();
        });
    }
}
```

Listing 3.1: Fluent API Modeling Override

3.2 Eager vs. Lazy Loading, Projections, and Pagination

Retrieving relational data efficiently is critical for system performance.

3.2.1 Eager Loading

Forces the loading of related entities inside the primary database query via internal or external SQL Joins.

```
var students = await _context.Students
                             .Include(s => s.Course)
                             .ToListAsync();
```

Listing 3.2: Eager Loading Query Example

3.2.2 Lazy Loading

Loads relational navigation properties on-demand when accessed. This can trigger the dangerous **$N + 1$ Query Problem**, where accessing a child property inside a loop ex-

ecutes N separate database queries.

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(connectionString)
           .UseLazyLoadingProxies());
```

Listing 3.3: Lazy Loading Proxies Setup

3.2.3 Query Projections

Projections decouple database structures from API outputs. Selecting specific fields rather than full entities prevents over-fetching and returns only the required data.

```
var studentPayloads = await _context.Students
    .Select(s => new StudentDTO
    {
        StudentId = s.StudentKey,
        StudentName = s.FullName,
        EnrolledCourse = s.Course.Title
    })
    .ToListAsync();
```

Listing 3.4: Optimal Select Projection

3.2.4 Server-Side Pagination & Filtering

To prevent database memory overload, query endpoints must paginate results on the database server before transmitting data.

```
public class PaginatedResult<T>
{
    public int PageNumber { get; set; }
    public int PageSize { get; set; }
    public int TotalCount { get; set; }
    public List<T> Data { get; set; }
}

public async Task<PaginatedResult<Employee>> GetPagedEmployeesAsync(
    string filter, string sortBy, bool isAscending, int pageNumber, int
    pageSize)
{
    var query = _context.Employees.AsQueryable();

    // 1. Filtering
    if (!string.IsNullOrEmpty(filter))
    {
        query = query.Where(e => e.Name.Contains(filter) || e.Email.
    Contains(filter));
    }

    // 2. Sorting
    if (!string.IsNullOrEmpty(sortBy))
    {
        query = isAscending ? query.OrderBy(e => EF.Property<object>(e,
    sortBy))
                            : query.OrderByDescending(e => EF.Property<
    object>(e, sortBy));
    }
```

```
26
27      // 3. Execution & Slicing
28      int totalRecords = await query.CountAsync();
29      var data = await query.Skip((pageNumber - 1) * pageSize)
30                            .Take(pageSize)
31                            .ToListAsync();
32
33      return new PaginatedResult<Employee>
34      {
35          PageNumber = pageNumber,
36          PageSize = pageSize,
37          TotalCount = totalRecords,
38          Data = data
39      };
40  }
```

Listing 3.5: Generic Server-Side Slicing Implementation

Chapter 4

UI Engineering with React

4.1 JSX Core Principles & Element Architecture

React replaces traditional template files with **JSX** (JavaScript XML), allowing developers to build declarative components. JSX expressions require strict adherence to key functional rules:

- **Single Parent Root:** All nested elements must reside inside one parent tag (or a React Fragment <> ... </>).
- **CamelCase Attributes:** Standard HTML properties are replaced with camelCase versions (e.g., `class` becomes `className`, `onclick` becomes `onClick`).
- **Strict Closing Tags:** Every element, including self-closing ones (e.g., `<br />` or `<img />`), must be explicitly terminated.

4.2 State, Immutability, and Custom Hooks

React states are **immutable**. You must never modify state variables directly. Always use setter methods with pure data updates to trigger component re-renders.

4.2.1 Safe State Management & Immutability Patterns

```
// Declaring State
const [user, setUser] = useState({ name: "Alice", age: 22 });
const [items, setItems] = useState([]);

// 1. Updating nested objects safely (using ES6 Spread operator)
setUser(prev => ({
    ...prev,
    age: 23
}));

// 2. Adding an item to an array (avoid push!)
setItems(prev => [...prev, "New Item"]);

// 3. Removing an item from an array
setItems(prev => prev.filter(item => item !== "Old"));
```

Listing 4.1: State Immutability Best Practices

4.2.2 Custom Hooks for Logic Reuse

Custom hooks abstract logic out of components, wrapping reactive functions into reusable utilities.

```
import { useState, useEffect } from 'react';

function useFetch(url) {
    const [data, setData] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

    useEffect(() => {
        let isMounted = true;
        setLoading(true);

        fetch(url)
            .then(res => res.json())
            .then(result => {
                if (isMounted) {
                    setData(result);
                    setLoading(false);
                }
            })
            .catch(err => {
                if (isMounted) {
                    setError(err);
                    setLoading(false);
                }
            });

        return () => { isMounted = false; };
    }, [url]);

    return { data, loading, error };
}
```

Listing 4.2: The useFetch Custom Hook Pattern

4.3 Global State with the Context API

Prop drilling is the anti-pattern of passing data down multiple nested child layers that do not require the value. The Context API solves this by managing global state.

The Context Paradigm

[State Provider] $\rightarrow$ Holds user state and exposes access directly to deep child elements.

$\rightarrow$ [Child View (reads user context directly without intermediary props)]

```
import React, { createContext, useState, useContext } from 'react';

const ThemeContext = createContext();

export function ThemeProvider({ children }) {
    const [theme, setTheme] = useState("light");
```

```
7
8       const toggleTheme = () => {
9           setTheme(prev => prev === "light" ? "dark" : "light");
10      };
11
12      return (
13          <ThemeContext.Provider value={{ theme, toggleTheme }}>
14              {children}
15          </ThemeContext.Provider>
16      );
17  }
18
19  export function useTheme() {
20      return useContext(ThemeContext);
21  }
```

Listing 4.3: Full Context API Architecture

Chapter 5

Testing & Diagnostic Automation

5.1 Unit Testing with NUnit and Mocking

Unit tests isolate and verify specific code paths in a system. NUnit is a popular, attribute-driven testing framework for .NET applications.

Table 5.1: NUnit Execution Lifecycle

Attribute	Purpose	Execution Timing
`[TestFixture]`	Designates a class containing test suites	Runs on setup initialization
`[SetUp]`	Initializes resources for individual runs	Executes before *each* test method
`[Test]`	Denotes a single test scenario	Runs sequentially
`[TestCase]`	Injects parameters for dynamic testing	Re-runs target method with new values
`[TearDown]`	Cleans up resource-intensive allocations	Executes after *each* test completes

5.1.1 Production-Grade NUnit Test Class

```
using NUnit.Framework;
using System;

[TestFixture]
public class CalculatorTests
{
    private Calculator _calculator;

    [SetUp]
    public void Init()
    {
        _calculator = new Calculator();
    }

    [Test]
    [TestCase(10, 20, 30)]
    [TestCase(-5, 5, 0)]
    public void Add_GivenArguments_ReturnsExpectedSum(int a, int b, int
    expected)
    {
        int actual = _calculator.Add(a, b);
        Assert.AreEqual(expected, actual);
    }
```

```
23
24      [Test]
25      public void Divide_ByZero_ThrowsDivideByZeroException()
26      {
27          Assert.Throws<DivideByZeroException>(() => _calculator.Divide(10,
        0));
28      }
29
30      [TearDown]
31      public void Cleanup()
32      {
33          _calculator = null;
34      }
35  }
```

Listing 5.1: Parameterized Mathematics Test Fixture

5.2 SonarQube Static Analysis & Quality Gates

SonarQube acts as an automated code reviewer, analyzing source files without executing them to detect bugs, vulnerabilities, and code smells.

5.2.1 The Quality Gate Paradigm

A **Quality Gate** is a set of boolean conditions that a code branch must satisfy before it can be merged into production. Typical requirements include:

- New Critical Bugs = 0
- Unit Test Code Coverage > 80%
- Security Vulnerabilities = 0
- Duplicated Lines of Code < 3%

5.2.2 Static Scanner Analysis Steps

To run an analysis against a .NET Web API project and export test coverage data to SonarQube:

```
1  # 1. Begin Scanner
2  dotnet sonarscanner begin /k:"CoreStudentApi" /d:sonar.host.url="http://
       localhost:9000" /d:sonar.token="sqp_token" /d:sonar.cs.vscoveragexml.
       reportsPaths="**/coverage.cobertura.xml"
3
4  # 2. Build the app
5  dotnet build --configuration Release
6
7  # 3. Execute unit tests with Cobertura code coverage output
8  dotnet test --collect:"XPlat Code Coverage"
9
10 # 4. End Scanner and submit reports to Server Dashboard
11 dotnet sonarscanner end /d:sonar.token="sqp_token"
```

Listing 5.2: SonarQube Command Line Execution

Chapter 6

Microservices & Continuous Pipelines

6.1 Microservices Architectural Mechanics

In a Microservices architecture, an application is split into small, independently deployable services that own their databases and communicate via lightweight protocols (e.g., HTTP/REST, gRPC, or AMQP queues).

Decoupled Service Model Flow
[Web Gateway] → Calls [Order Service (DB: SQL Server)] → [Product Service (DB: MongoDB)]

6.1.1 Resilient Synchronous Communication

To make synchronous service-to-service communication resilient, implement failure fallbacks when a dependent service is unavailable.

```
using System;
using System.Net.Http;
using System.Net.Http.Json;
using System.Threading.Tasks;

public class ResilientOrderProcessor
{
    private readonly HttpClient _client;

    public ResilientOrderProcessor(HttpClient client)
    {
        _client = client;
    }

    public async Task<ProductPayload> FetchProductInfoAsync(int id)
    {
        try
        {
            // Set brief timeout
            _client.Timeout = TimeSpan.FromSeconds(3);
            return await _client.GetFromJsonAsync<ProductPayload>($"api/
   products/{id}");
        }
        catch (Exception ex)
```

```
        {
            // Graceful Fallback
            return new ProductPayload
            {
                Id = id,
                Name = "Temporary Fallback Product (Product Service Down)",
                Price = 0.00m
            };
        }
    }
}
```

Listing 6.1: Resilient Http Client with Local Mock Fallback

6.2 Enterprise DevOps: Continuous Pipelines

Continuous Integration (CI) and Continuous Delivery (CD) automate validation and deployment workflows. High-performance projects use a declarative 'Jenkinsfile' pipeline as code.

```
pipeline {
    agent any

    tools {
        maven 'Maven_3.x'
        jdk 'Java_17'
    }

    stages {
        stage('Pull Source') {
            steps {
                git branch: 'main', url: 'https://github.com/enterprise/gateway-api.git'
            }
        }
        stage('Lint & Security Scan') {
            steps {
                echo 'Checking for dependency updates and vulnerabilities...'
            }
        }
        stage('Compile & Test') {
            steps {
                sh 'mvn clean test'
            }
        }
        stage('SonarQube Quality Gate Check') {
            steps {
                echo 'Validating build code quality criteria against dashboard parameters...'
            }
        }
        stage('Publish Artifacts') {
            steps {
                archiveArtifacts artifacts: 'target/*.jar', fingerprint: true
            }
```

```
            }
        }
}
```

Listing 6.2: Declarative CI/CD Jenkinsfile Pipeline

6.3 Azure Cloud Service Topology

Enterprise applications run across virtual boundaries in the cloud. Microsoft Azure provides specific service layers depending on hosting requirements:

API Management (APIM): Secures endpoints, throttles traffic, and aggregates APIs under a single gateway domain.

Azure Functions (Serverless): Runs event-driven code blocks without the overhead of hosting web servers.

Azure Service Bus: Decouples microservices using highly reliable queuing networks.

Azure Container Instances (ACI): Runs serverless Docker containers instantly without managing virtual machines.

Azure Kubernetes Service (AKS): Manages cluster orchestration, load balancing, and scaling of containerized microservices in production.